Catching Campaigns, Not Connections: Coordination-Aware Hypergraph Networks for Multi-Stage Intrusion Detection

Anonymous Authors
Institution

Abstract

Provenance-based intrusion detection systems analyze kernel-level audit logs to identify Advanced Persistent Threats, but existing approaches operate at coarse granularities — classifying entire graphs (KAIROS, MAGIC) or individual nodes (FLASH, ThreaTrace) — without attributing detection to specific attack events. We present HyperMamba, a system that detects individual crossprocess attack events in real-time audit streams by maintaining persistent entity state via Selective State Space Models. HyperMamba models each audit event as a temporal hyperedge connecting two or three system entities, preserving the native variable-arity structure of the Common Data Model. A Selective SSM maintains a state vector for every entity, accumulating behavioral context across the entire stream; at each event, attention-based hyperedge aggregation combines participating entities' states, the SSM updates each entity, and a classifier produces an event-level detection decision. Ablation studies reveal a clear hierarchy of component contributions: gated process identity modulation is the dominant mechanism for cross-campaign generalization (-55% event AUPRC when removed), cross-entity hyperedge aggregation provides the primary structural contribution (-8.1%), and persistent state adds residual discrimination for the stealthiest events (-4.6%). Removing both state and process identity collapses detection to near-random (0.20 AUPRC), confirming that isolated event features cannot discriminate crossprocess attacks. On the DARPA TC Engagement 3 datasets, HyperMamba achieves event-level AUPRC of 0.76 on Theia (cross-campaign, crossprocess label) and 0.97 on CADETS (cross-campaign, broad label), with FPR ≤ 0.0011, while processing events at $194\times$ the peak audit ingestion rate at inference on a single GPU.

1 Introduction

Provenance-based Network Intrusion Detection Systems (NIDS) analyze kernel-level audit logs to detect Advanced Persistent Threats (APTs) on monitored hosts. Recent state-of-the-art systems — KAIROS [3], MAGIC [7], FLASH [8], and ThreaTrace [9] — have demonstrated strong detection capabilities. However, these systems operate at fundamentally coarse granularities: KAIROS and MAGIC assign anomaly scores to entire provenance graphs, while FLASH and ThreaTrace classify individual nodes (processes or files). None produce *event-level* classifications. When a security analyst receives a node-level alert flagging a process as malicious, the analyst must still manually inspect thousands of that process's events to identify which specific operations constituted the attack. This forensic gap between detection and attribution limits the operational utility of existing systems.

The granularity gap is not merely inconvenient — it reflects a structural limitation. Crossprocess attack events — a label we define in Section 3.4 to denote events by child processes spawned during multi-stage APT campaigns, excluding the initial entry-point process — use the same system calls as benign background activity. Their per-event feature distributions satisfy $P(\mathbf{x}_i \mid \text{attack}) \approx P(\mathbf{x}_i \mid \text{benign})$, making them statistically indistinguishable in isolation. A system that evaluates events independently cannot reliably separate malicious reads and writes from benign ones. The discriminative signal exists only in the *accumulated behavioral history* of the acting entity: the sequence of prior operations that reveals the entity's role in an ongoing campaign. Existing systems discard this signal by operating on static graph snapshots without persistent entity state.

We present **HyperMamba**, a provenance-based NIDS that detects individual crossprocess attack events in real-time audit streams by maintaining persistent entity state via Selective State Space Models. HyperMamba models each audit event as a temporal hyperedge connecting two or three system entities, preserving the native variable-arity structure of the Common Data Model (CDM). A Selective SSM maintains and updates a state vector for every entity in the system, accumulating behavioral context across the entire event stream. At each event, the model aggregates the participating entities' states via attention-based hyperedge aggregation, updates each entity's state through the SSM recurrence, and classifies the event using both the updated states and the event's own features.

The result is event-level, real-time detection with per-event forensic attribution.

Our contributions are:

1. **Provenance hypergraph formalization (C1).** We formalize audit streams as temporal hypergraphs with variable-arity hyperedges and demonstrate via controlled ablation that cross-entity state aggregation within hyperedges provides 8.1% AUPRC improvement over isolated per-entity tracking.

2. **Semantic process identity and temporal state as complementary detection mechanisms (C2).** We prove empirically that the combination of semantic process identity and temporal entity state is jointly necessary for crossprocess attack detection. Removing both collapses event AUPRC from 0.76 to 0.20 on the cross-campaign evaluation. Process identity is the dominant single contributor (-55% when removed), while accumulated state provides residual discrimination for the stealthiest events (-4.6%). Neither alone is sufficient (Section 5.3).

3. **Gated process identity for cross-campaign generalization (C3).** We introduce a gated process-path modulation mechanism that provides campaign-invariant semantic anchors. Removing process identity while retaining the full state architecture reduces cross-campaign AUPRC from 0.76 to 0.34 (Section 5.3).

4. **Event-level detection at operational scale (C4).** HyperMamba achieves event-level AUPRC of 0.76 (cross-campaign) with FPR of 0.0011, while processing $194\times$ the peak audit ingestion rate at inference on a single GPU (Section 5.2).

Section 2 introduces the provenance data model and SSM background. Section 3 defines the threat model and detection task. Section 4 details the HyperMamba architecture. Section 5 presents the evaluation. Sections 7 and 6 discuss related work and limitations.

2 Background

This section defines the three concepts required to follow the architecture (Section 4): the provenance data model, its formalization as a temporal hypergraph, and the Selective State Space Model recurrence.

2.1 Provenance Data Model

Modern host-based audit systems based on the DARPA Common Data Model (CDM) record every kernel-mediated operation as a typed event linking a *subject* (the acting process) to one or two *objects* (the targets of the operation). A file read generates an event with one subject and one object (`process` $\rightarrow$ `file`); a memory mapping of a shared library generates an event with one subject and two objects (`process` $\rightarrow$ `file` $\rightarrow$ `memory_object`). Each event carries metadata: a discrete event type (one of $\sim$40 CDM types such as `EVENT_READ`, `EVENT_MMAP`), continuous attributes (byte count, flags), and a nanosecond-resolution timestamp. Over a multi-day engagement, a single host generates tens of millions of such events.

2.2 Provenance Hypergraph

We formalize the audit stream as a temporal hypergraph.

Definition 1 (Provenance Hypergraph)**.** *A* provenance hypergraph *is a tuple $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a finite set of system entities (processes, files, sockets, memory objects) and $\mathcal{E} = \{e_1, e_2, \dots\}$ is a chronologically ordered sequence of hyperedges. Each hyperedge $e_t = (v_{subj}, v_{obj}, v_{obj2}, \tau_t, \mathbf{a}_t)$ connects $|V_{e_t}| \in \{2, 3\}$ entities, where v_{obj2} is absent for size-2 events, τ_t is the event timestamp, and $\mathbf{a}_t$ is the attribute vector.*

For example, when the compromised Firefox process maps the dropped binary `malicious.so` into executable memory, the resulting `EVENT_MMAP` hyperedge connects three entities: Firefox (v_{subj}), the file `malicious.so` (v_{obj}), and the allocated memory region (v_{obj2}).

Why hyperedges, not pairwise edges. Prior provenance-based NIDS [3, 7–9] decompose each multi-object event into independent pairwise edges, discarding the co-occurrence structure. In the DARPA TC E3 Theia dataset, 2.8% of events are size-3 hyperedges. While this fraction appears small, size-3 events are $3.0\times$ enriched among crossprocess attack events relative to benign events, driven primarily by `EVENT_MMAP` operations (e.g., a compromised process mapping a malicious shared library into executable memory). This enrichment motivates retaining the native hyperedge arity rather than decomposing into pairwise interactions. Figure 1 illustrates this with a concrete example from the Theia attack campaign.

2.3 Selective State Space Models

State Space Models (SSMs) define a linear recurrence over a latent state $\mathbf{h} \in \mathbb{R}^n$, mapping an input sequence $x(t)$ to an output $y(t)$ through continuous-time dynamics:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}\mathbf{h}(t) \tag{1}$$

where $\mathbf{A} \in \mathbb{R}^{n\times n}$ governs state decay and $\mathbf{B} \in \mathbb{R}^{n\times 1}$ controls input injection. For discrete sequences, the continuous dynamics are discretized with a step size Δ to yield:

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \quad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\bar{\mathbf{A}} - \mathbf{I}) \cdot \Delta\mathbf{B} \tag{2}$$

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \quad y_t = \mathbf{C}\mathbf{h}_t \tag{3}$$

The *Selective* SSM [5] makes $\mathbf{B}$, $\mathbf{C}$, and Δ input-dependent, allowing the model to selectively retain or discard information at each step based on the current input. In Section 4.5, we adapt this mechanism to operate on entity states within a hypergraph, where Δ is further conditioned on the elapsed time since the entity was last observed.

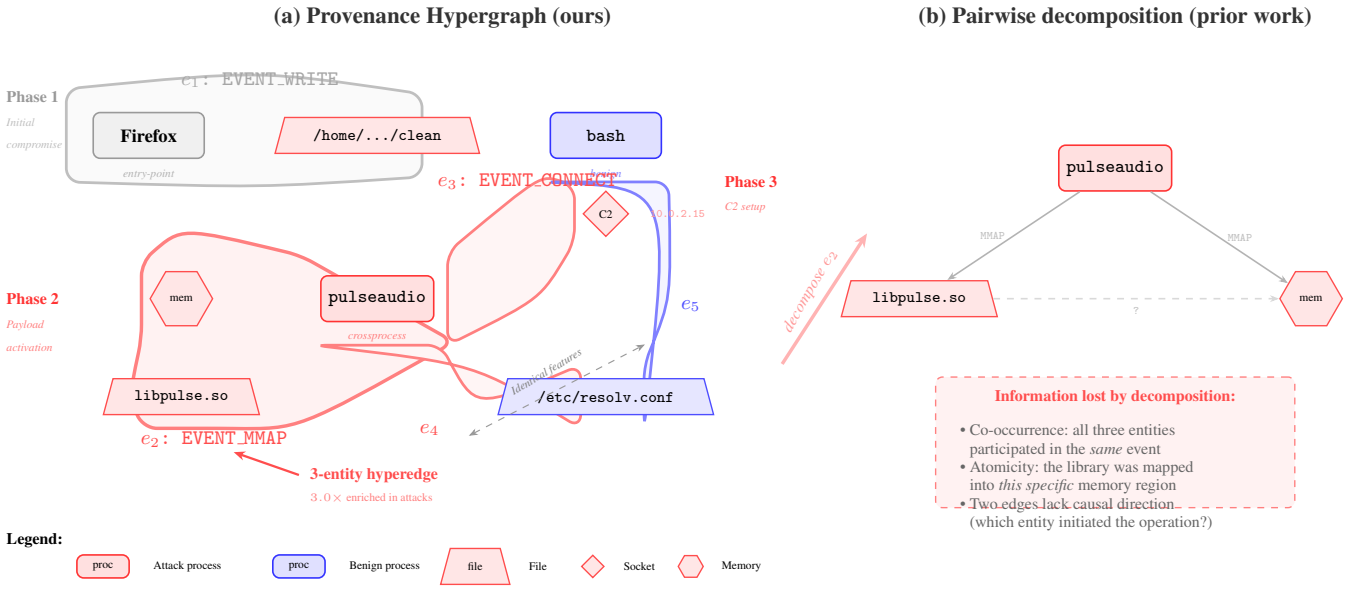

Figure 1: Provenance hypergraph from the DARPA TC E3 Theia attack campaign. **(a)** Each audit event is a temporal hyperedge connecting 2–3 system entities. The `EVENT_MMAP` hyperedge e_2 natively connects three entities (process, file, memory region) as a single atomic operation. Events e_4 (attack) and e_5 (benign) have identical per-event features—both read `/etc/resolv.conf` with the same byte count—but are distinguishable through the accumulated state of their subject entities. Gray shading marks the entry-point event (excluded from the crossprocess label). **(b)** Prior systems decompose multi-object events into independent pairwise edges, discarding co-occurrence structure and causal atomicity. Size-3 events are 3.0× enriched among crossprocess attack events.

3 Threat Model

3.1 Attacker Model

We consider an Advanced Persistent Threat (APT) adversary executing multi-stage crossprocess attack campaigns on a monitored host.

Knowledge. The attacker is aware that kernel-level audit logging is active on the target host. In the primary evaluation, the attacker does not have knowledge of the detection model's architecture or parameters (black-box setting). We discuss the implications of a white-box attacker who targets the SSM state propagation mechanism in Section 6.

Capabilities. The attacker can execute arbitrary operations through standard system APIs: spawning processes, reading and writing files, mapping memory regions, and establishing network connections. All such operations are recorded as audit events by the kernel's logging infrastructure. The attacker *cannot* tamper with or suppress kernel-level audit logs. This assumption is standard in the provenance-based NIDS literature and reflects deployments where audit collection runs at a higher privilege level than user-space processes.

Goal. The attacker's objective is to execute a complete multi-stage crossprocess campaign—initial exploitation of an entry-point application, deployment of secondary tooling via child processes, establishment of command-and-control channels, and data exfiltration—while evading event-level detection. The attacker achieves evasion if the individual system events generated by the campaign's child processes are not flagged as malicious by the detection system.

3.2 DARPA TC E3 Attack Instantiation

We ground the threat model in a specific multi-stage campaign from the DARPA TC Engagement 3 Theia dataset. The attack proceeds through four distinct phases, each producing audit events in the host's kernel-level log:

1. **Initial compromise.** The attacker exploits a vulnerability in the Firefox browser process. Firefox executes an `EVENT_WRITE` to drop a malicious binary at `/home/admin/clean`. This single event is the *entry point*—Firefox is the compromised process.

2. **Payload activation.** Firefox spawns the dropped payload, which in turn launches downstream processes including `pulseaudio` and `gconf-helper` via `EVENT_EXECUTE`. These child processes load shared libraries (`EVENT_MMAP`) and read system configuration files (`EVENT_READ` on `/etc/resolv.conf`, `/etc/hosts`).

3. **Command and control.** The spawned processes establish reverse shell connections to the attacker's infrastruc-

ture via EVENT_CONNECT to an external IP address. Subsequent EVENT_SENDTO and EVENT_RECVFROM events carry C2 commands and responses.

4. **Exfiltration.** Under C2 direction, the child processes read sensitive files (EVENT_READ) and transmit their contents to the attacker's server (EVENT_SENDTO). Additional processes may be spawned for lateral reconnaissance.

Crossprocess label definition. In this paper, the *crossprocess* label refers to the events executed by the child processes spawned downstream of the entry-point process—in the Theia dataset, these are instances of `pulseaudio` and `gconf-helper` and any processes they subsequently spawn. The entry-point process itself (Firefox) is *excluded* from this label. This exclusion is deliberate: the initial exploit of Firefox may produce anomalous system calls (e.g., a browser writing an executable binary), which are detectable by simpler, stateless methods. The harder detection problem—and the one HyperMamba targets—is identifying the later-stage child processes whose individual events are operationally indistinguishable from benign system activity.

3.3 Why Crossprocess Attacks Are Hard for Stateless Detection

Crossprocess attacks are executed by *child processes* that the compromised entry-point process spawns to carry out the operational stages of the campaign—reconnaissance, lateral movement, data staging, and exfiltration. These child processes (e.g., `pulseaudio` instances spawned by the dropped payload, or injected shell scripts) use precisely the same system calls as benign background processes: EVENT_READ on configuration files, EVENT_WRITE to temporary directories, EVENT_MMAP for shared library loading, and EVENT_CONNECT to external hosts. A stateless classifier, which evaluates each event in isolation, observes these calls without historical context and faces a fundamental ambiguity.

Concrete example. Consider two events drawn from the DARPA TC E3 Theia dataset:

1. `bash` (benign) executes EVENT_READ on `/etc/resolv.conf`, transferring 242 bytes, $\Delta t = 0.4$s since its last event.

2. `pulseaudio` (crossprocess attack) executes EVENT_READ on `/etc/resolv.conf`, transferring 242 bytes, $\Delta t = 0.3$s since its last event.

Both events share the same event type, the same target file, and effectively identical continuous features (byte count, flags, inter-arrival time). A feature vector extracted from either event in isolation is statistically indistinguishable. More formally, the per-event feature distributions satisfy $P(\mathbf{x}_i \mid \text{attack}) \approx P(\mathbf{x}_i \mid \text{benign})$ for the vast majority of crossprocess attack events. The discriminative signal is not in the event itself, but in the *accumulated behavioral history* of the subject entity: `pulseaudio` was spawned by a compromised process chain originating from Firefox, previously established a C2 connection, and has been writing to staging directories—a pattern visible only through persistent state.

Empirical validation. The ablation study in Section 5.3 provides direct empirical evidence for this stealthiness condition. Removing both accumulated entity state and semantic process identity from HyperMamba (while retaining the event encoder, hyperedge aggregator, and classifier) causes the cross-campaign test AUPRC to collapse from 0.76 to 0.20—near random. Crossprocess attack events are, by construction, not identifiable from isolated event features (event type and continuous quantities) alone.

3.4 Detection Task

Given a continuous chronological stream of audit events $e_1, e_2, \ldots$ generated by a single monitored host, the detection task is to classify each event e_t as either benign ($y_t = 0$) or crossprocess-attack ($y_t = 1$) at event arrival time. Formally, the detector is a function:

$$f_\theta(e_t \mid e_1, \ldots, e_{t-1}) \to [0, 1] \tag{4}$$

that produces a probability $\hat{y}_t$ using the current event e_t as input and all previously observed events $e_1, \ldots, e_{t-1}$ as historical context. The detector *cannot* access future events $e_{t+1}, e_{t+2}, \ldots$ — the classification must be strictly causal.

Scope. This work addresses host-based detection from kernel-level audit logs. Network-flow-based intrusion detection, payload inspection, and signature-based matching are outside the scope of this paper.

4 The HyperMamba Architecture

HyperMamba processes audit events in strict chronological order as a continuous stream. Each event is a temporal hyperedge connecting two or three system entities (Section 2.2). For each incoming event, the model executes a five-stage forward pass that reads and updates persistent entity states. Figure 2 summarizes the pipeline; Sections 4.2–4.7 detail each stage.

4.1 Overview

Let e_t denote the audit event arriving at discrete step t, with participating entities $V_{e_t} \subseteq \mathcal{V}$, where $|V_{e_t}| \in \{2, 3\}$. We denote the three entity slots as v_{subj} (subject process), v_{obj} (primary object), and v_{obj2} (secondary object, absent for size-2 events). Each entity $v \in \mathcal{V}$ maintains a persistent state vector $\mathbf{s}_v \in \mathbb{R}^d$ in the `EntityStateBank`, where $d = 256$ throughout this paper.

HyperMamba processes e_t in five stages:

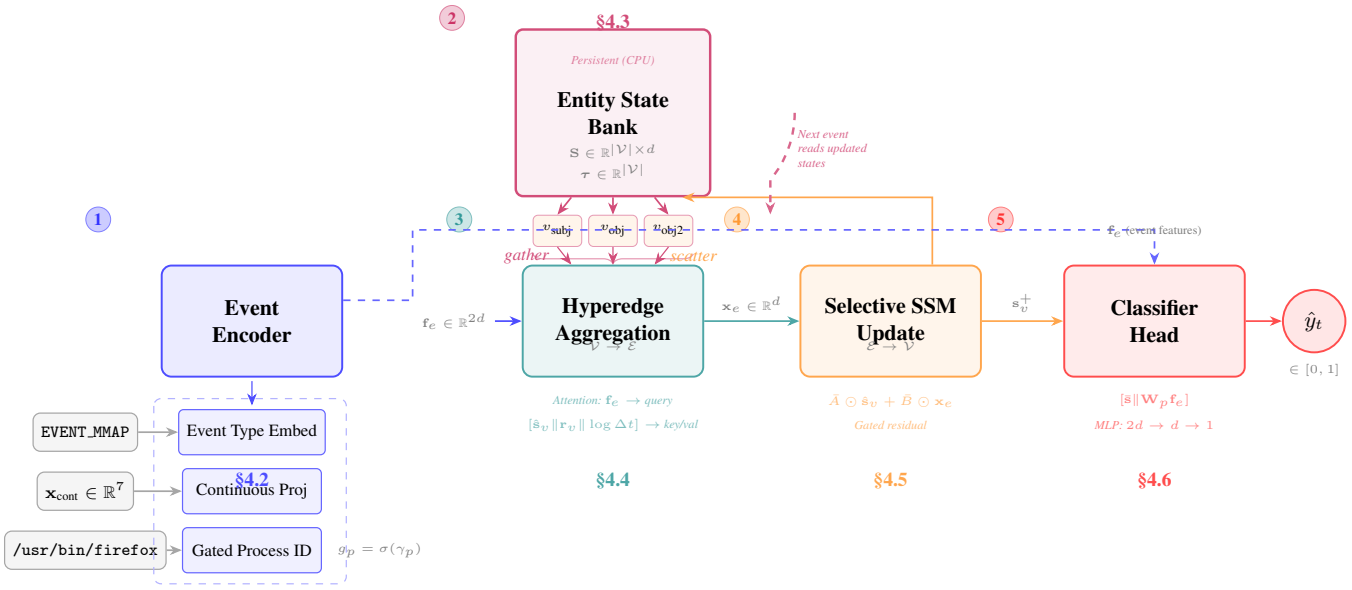

Figure 2: HyperMamba architecture. Each audit event e_t is processed in five stages. **(1)** The event encoder combines event type, continuous features, and gated process identity into $\mathbf{f}_e$. **(2)** Entity states $\hat{\mathbf{s}}_v$ and elapsed times Δt_v are gathered from the persistent `EntityStateBank`. **(3)** Attention-based hyperedge aggregation ($\mathcal{V} \to \mathcal{E}$) fuses entity states conditioned on $\mathbf{f}_e$. **(4)** A Selective SSM updates each entity's state ($\mathcal{E} \to \mathcal{V}$) and scatters results back to the bank. **(5)** Post-update states and event features are concatenated and classified. The bank persists across events, creating a recurrent information pathway across entities and time.

1. **Event Encoding** (Section 4.2). The event type, continuous features (byte count, flags, temporal inter-arrival statistics), and the subject's process path are projected and combined into a dense event representation $\mathbf{f}_e \in \mathbb{R}^{2d}$. Process path identity is injected via a gated additive residual with training-time identity dropout to prevent per-process memorization (Section 4.2).

2. **Entity State Retrieval** (Section 4.3). For each entity $v \in V_{e_t}$, the current state $\mathbf{s}_v$ and elapsed time Δt_v since v was last observed are read from the `EntityStateBank`. A validity mask m_v marks absent secondary objects.

3. **Hyperedge Aggregation**, $\mathcal{V} \to \mathcal{E}$ (Section 4.4). The retrieved entity states are aggregated into a single hyperedge representation $\mathbf{x}_e \in \mathbb{R}^d$ via scaled dot-product attention, where event features $\mathbf{f}_e$ serve as the query and the entity states — concatenated with learned role embeddings $\mathbf{r}_v$ and log-scaled elapsed time $\log(1 + \Delta t_v)$ — serve as keys and values.

4. **SSM State Update**, $\mathcal{E} \to \mathcal{V}$ (Section 4.5). Each participating entity's state $\mathbf{s}_v$ is updated via a Selective State Space Model. The update is role-specific (subjects and objects receive asymmetric updates), time-aware (the discretization step Δ_i incorporates Δt_v), and gated (a learned residual gate controls how much of the SSM output replaces the prior state).

5. **Classification** (Section 4.6). The post-update entity states are mean-pooled (masked by validity) and concatenated with $\mathbf{f}_e$ to produce a scalar logit $\hat{y}_t = \sigma(\text{MLP}([\bar{\mathbf{s}} \,\|\, \mathbf{W}_p \mathbf{f}_e]))$, where $\hat{y}_t \in [0, 1]$ is the probability that e_t is a crossprocess attack event.

The state written to the bank at stage 4 for entity v is the state read at stage 2 the next time v participates in any event. This creates a recurrent information pathway across entities and across time, with memory bounded at $\mathcal{O}(|\mathcal{V}| \times d)$ independent of the number of events processed. Training uses Truncated Backpropagation Through Time with detachment at chunk boundaries (Section 4.7).

4.2 Event Encoding and Process Identity Modulation

Each audit event e_t carries three categories of raw input: a discrete event type (one of ∼40 CDM event types such as `EVENT_READ`, `EVENT_MMAP`, `EVENT_CONNECT`), a vector of continuous features $\mathbf{x}_{\text{cont}} \in \mathbb{R}^{n_c}$ (byte count, flags, temporal inter-arrival statistics), and the filesystem path of the subject process (e.g., `/usr/bin/bash`, `/home/admin/clean`). The event encoder projects these into a dense representation $\mathbf{f}_e \in \mathbb{R}^{2d}$ that serves as input to all downstream stages.

Event type and continuous features. The event type is mapped to a learned embedding $\mathbf{e}_{\text{type}} = \text{Embed}_{\text{type}}(\text{type}(e_t)) \in \mathbb{R}^d$. The continuous features are projected via a linear layer $\mathbf{c} = \mathbf{W}_c \mathbf{x}_{\text{cont}} + \mathbf{b}_c \in \mathbb{R}^d$.

Gated process identity modulation. Integer entity identifiers assigned during preprocessing have no semantic content: the same malware binary `/home/admin/clean` receives different integer IDs in different campaigns because the entity vocabulary is constructed per-dataset. To provide a semantic signal that generalizes across campaigns, we embed the subject's process path via a learned lookup $\mathbf{p}_{\text{raw}} = \text{Embed}_{\text{proc}}(\text{path}(v_{\text{subj}})) \in \mathbb{R}^{64}$ from a vocabulary of known process names (defined in Section 5.1). This is projected to model dimension $\mathbf{p} = \mathbf{W}_p \mathbf{p}_{\text{raw}} \in \mathbb{R}^d$.

Process identity is injected as a *gated additive residual* on the event type embedding rather than via concatenation:

$$\mathbf{e}_{\text{mod}} = \mathbf{e}_{\text{type}} + g_p \cdot \mathbf{p} \tag{5}$$

where $g_p = \sigma(\gamma_p)$ is a scalar gate controlled by a single learnable parameter γ_p, initialized to -2.0 so that $g_p \approx 0.12$ at the start of training. Process identity is injected into the event type embedding rather than the continuous features because the process path modulates event semantics (e.g., an `EVENT_MMAP` by `firefox` carries different contextual meaning than one by `pulseaudio`), whereas continuous features measure process-invariant quantities like byte counts. The near-zero gate initialization forces the model to first learn from event-type and continuous features before the process identity signal is mixed in. Without gating, the additive process signal is unconstrained at initialization, creating a gradient shortcut through process identity that bypasses event-level discrimination. The necessity of process identity is confirmed empirically in Section 5.3: removing the process embedding while retaining the full state architecture reduces cross-campaign AUPRC from 0.76 to 0.34.

Identity dropout. During training, each event's process identity is independently replaced with an unknown token (index 0) with probability $\rho = 0.3$:

$$\text{path}(v_{\text{subj}}) \leftarrow \begin{cases} \text{path}(v_{\text{subj}}) & \text{with probability } 1-\rho \\ \texttt{<unk>} & \text{with probability } \rho \end{cases} \tag{6}$$

This prevents the classifier from relying on process identity as a sufficient detection signal. The model must maintain a working detection pathway through event-type and entity-state features alone, using process identity as a supplementary modulation rather than a shortcut.

Final event representation. The three components are combined and normalized:

$$\mathbf{f}_e = \text{LayerNorm}([\mathbf{e}_{\text{mod}} \parallel \mathbf{c}]) \in \mathbb{R}^{2d} \tag{7}$$

where $\parallel$ denotes concatenation. The event type (modulated by process identity) occupies the first d dimensions; continuous features occupy the second d dimensions.

4.3 Entity State Bank

The `EntityStateBank` stores two tensors — a state matrix $\mathbf{S} \in \mathbb{R}^{|\mathcal{V}| \times d}$, where row $\mathbf{S}[v]$ holds the current state $\mathbf{s}_v$ of entity v, and a last-seen timestamp vector $\boldsymbol{\tau} \in \mathbb{R}^{|\mathcal{V}|}$, where $\boldsymbol{\tau}[v]$ records the most recent time entity v participated in any event — in **CPU system RAM** with page-locked (pinned) memory. The tensors are deliberately not registered as PyTorch buffers, so `model.to(device)` does not transfer them to the GPU. This design enables scaling to entity populations that would exhaust GPU VRAM (e.g., 176M raw entities in the TRACE dataset before re-indexing). At each chunk, only the $\sim 3C$ entity states actually touched by the chunk's events (where C is the chunk size) are gathered to the GPU via non-blocking pinned-memory transfers, processed through the SSM, and scattered back. Both tensors are initialized to zero (states) and -1 (timestamps, indicating unseen) at the start of each training epoch and persist across event chunks within the epoch. Resetting at epoch boundaries prevents the model from overfitting to the specific state trajectory accumulated in earlier epochs, while intra-epoch persistence provides the long-range context required by the thesis.

Memory bound. The bank requires $|\mathcal{V}| \times (d+1)$ floating-point values in system RAM. For the Theia dataset ($|\mathcal{V}| \approx 7.1 \times 10^6$, $d = 256$), this amounts to 7.3 GB — well within the 64 GB system RAM of a commodity server, and independent of GPU VRAM constraints. The per-chunk GPU transfer involves at most $3C$ entity states (e.g., $3 \times 16{,}384 = 49{,}152$ vectors, $\sim$48 MB at $d = 256$), which is negligible relative to GPU memory.

State retrieval and elapsed time. For an incoming event e_t with entity slots $(v_{\text{subj}}, v_{\text{obj}}, v_{\text{obj2}})$, the model gathers a state tensor $\mathbf{S}_e \in \mathbb{R}^{3 \times d}$ by indexing into $\mathbf{S}$. A validity mask $\mathbf{m} \in \{0,1\}^3$ is set to 0 for absent entities: size-2 hyperedges store $v_{\text{obj2}} = -1$, which is clamped to index 0 during retrieval. The mask ensures that the spuriously read state at index 0 contributes zero to both the aggregation computation and the write-back path, preventing corruption of entity 0's state.

The elapsed time for each entity is computed as:

$$\Delta t_v = \max(0,\ t_{\text{curr}} - \boldsymbol{\tau}[v]) \tag{8}$$

where $\Delta t_v = 0$ for previously unseen entities ($\boldsymbol{\tau}[v] = -1$). The floor at zero guards against non-monotonic timestamps from intra-shard reordering. All timestamps are converted to seconds relative to a global reference t_0 (the nanosecond timestamp of the first event in the training set):

$$t_{\text{curr}} = (\text{timestamp\_nanos}(e_t) - t_0) \ /\ 10^9 \tag{9}$$

All splits (train, validation, test) share the same t_0 so that Δt_v remains valid across the train-to-test boundary during warm-start evaluation (Section 4.7).

The elapsed time is passed to downstream stages in log-scaled form as $\log(1 + \Delta t_v)$ to compress the dynamic range: inter-event gaps in the Theia dataset span from microseconds to hours.

Hub-entity normalization. High-degree entities (e.g., `init` with PID 1, which participates in over 1.5×10^5 events

in the Theia dataset) accumulate state vectors with unbounded norms through repeated SSM updates. Without normalization, the key projections in the aggregation stage (Section 4.4) produce attention scores that overflow `float32` softmax ($\exp(x)$ for $x > 88$). To prevent this, retrieved states are normalized via layer normalization before entering the aggregation stage. This normalization is applied only to the retrieved copy $\hat{\mathbf{S}}_e$; the bank $\mathbf{S}$ retains unnormalized states:

$$\hat{\mathbf{S}}_e = \text{LayerNorm}(\mathbf{S}_e) \odot \mathbf{m} \tag{10}$$

The validity mask $\mathbf{m}$ is applied after normalization to ensure absent entity slots contribute zero to downstream computations. All subsequent references to entity states within a single forward pass refer to $\hat{\mathbf{S}}_e$.

State write-back. After the SSM state update (Section 4.5), the updated states are scattered back into $\mathbf{S}$ via in-place indexed assignment, and $\boldsymbol{\tau}$ is updated with t_{curr}. Only valid entities (those with $\mathbf{m}_v = 1$) are written. To prevent numerical corruption from propagating across chunks, NaN values in the updated states are replaced with zero before write-back.

4.4 Hyperedge Aggregation ($\mathcal{V} \to \mathcal{E}$)

Given the normalized entity states $\hat{\mathbf{S}}_e$ (Equation 10) and the event representation $\mathbf{f}_e$ (Equation 7), the aggregation stage produces a single hyperedge representation $\mathbf{x}_e \in \mathbb{R}^d$ that fuses the states of the 2–3 participating entities, conditioned on the event semantics. The aggregation determines *how much* each entity's accumulated history contributes to the current event's representation, which directly controls the information available to the classifier.

Role embeddings. Each entity slot is assigned a learned role embedding $\mathbf{r}_v \in \mathbb{R}^d$ from a 3-entry lookup table (Subject, Primary Object, Secondary Object). Role embeddings are necessary because the aggregation operates on an unordered set of states; without positional role information, the attention mechanism cannot distinguish the subject from the object, discarding the directed structure of the event.

Entity descriptors. For each entity $v \in V_{e_t}$, we construct a descriptor $\mathbf{h}_v$ by concatenating the normalized state, role embedding, and log-scaled elapsed time:

$$\mathbf{h}_v = [\hat{\mathbf{s}}_v \parallel \mathbf{r}_v \parallel \log(1 + \Delta t_v)] \in \mathbb{R}^{2d+1} \tag{11}$$

Dynamic attention. The event features $\mathbf{f}_e$ serve as the query, and the entity descriptors serve as keys and values. This conditions the aggregation on the event type: for an `EVENT_WRITE`, the object (file being written to) may carry more discriminative state than the subject, while for an `EVENT_MMAP`, the secondary object (memory region) carries the signal identified in Section 2.2. The attention computation is:

$$\mathbf{q} = \mathbf{W}_q \mathbf{f}_e \in \mathbb{R}^d \tag{12}$$

$$\mathbf{k}_v = \mathbf{W}_k \mathbf{h}_v \in \mathbb{R}^d \tag{13}$$

$$\mathbf{v}_v = \mathbf{W}_v \mathbf{h}_v \in \mathbb{R}^d \tag{14}$$

where $\mathbf{W}_q \in \mathbb{R}^{d \times 2d}$, $\mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d \times (2d+1)}$. Attention weights are computed via masked scaled dot-product. For each entity slot v, the raw score is:

$$s_v = \mathbf{q}^\top \mathbf{k}_v \,/\, \sqrt{d} \tag{15}$$

Invalid slots ($\mathbf{m}_v = 0$) have their scores set to $-\infty$ before softmax, so that $\exp(-\infty) = 0$ in both numerator and denominator:

$$\alpha_v = \begin{cases} \dfrac{\exp(s_v)}{\sum_{u:\, \mathbf{m}_u = 1} \exp(s_u)} & \text{if } \mathbf{m}_v = 1 \\ 0 & \text{if } \mathbf{m}_v = 0 \end{cases} \tag{16}$$

As a numerical safeguard, any NaN values in the attention weights (which can arise from degenerate events where all slots are masked, producing $0/0$ in softmax) are replaced with zero. The hyperedge representation is then:

$$\mathbf{x}_e = \mathbf{W}_{\text{out}} \sum_{v \in V_{e_t}} \alpha_v \, \mathbf{v}_v \in \mathbb{R}^d \tag{17}$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times d}$ is a linear output projection. The role embeddings $\mathbf{r}_v$ are retained and passed to the SSM update stage (Section 4.5), where they ensure role-specific state propagation.

4.5 Selective SSM State Update ($\mathcal{E} \to \mathcal{V}$)

Given the hyperedge representation $\mathbf{x}_e$ (Equation 17) and the role embeddings $\mathbf{r}_v$ from the aggregation stage, the SSM updater computes a new state $\mathbf{s}_v^+$ for each participating entity $v \in V_{e_t}$. This is the mechanism through which accumulated behavioral context propagates across entities and across time: the updated state encodes both the entity's prior history (via the decay term) and the current event's contribution (via the input term).

State decay matrix. The diagonal state decay matrix $A \in \mathbb{R}^d$ is stored in log-space as a learnable parameter $\mathbf{A}_{\log} \in \mathbb{R}^d$ and recovered as:

$$A = -\exp(\text{clamp}(\mathbf{A}_{\log}, -4, 4)) \tag{18}$$

All entries of A are strictly negative, ensuring that states decay over time in the absence of new input. $\mathbf{A}_{\log}$ is initialized with $\log(\text{linspace}(0.1, 2.0, d))$, producing initial decay rates that span $A_j \in [-2.0, -0.1]$. This multi-timescale initialization assigns different state dimensions different decay rates: slow-decaying dimensions ($A_j \approx -0.1$) retain information

over many events for long-range dependencies, while fast-decaying dimensions ($A_j \approx -2.0$) respond to recent activity. Clamping $\mathbf{A}_{\log}$ to $[-4, 4]$ prevents numerical overflow during exponentiation.

Role-specific input projection. The input matrix B is computed from the concatenation of the hyperedge representation $\mathbf{x}_e$ and the entity's role embedding $\mathbf{r}_v$:

$$B_v = \mathbf{W}_B[\mathbf{x}_e \parallel \mathbf{r}_v] \in \mathbb{R}^d \tag{19}$$

where $\mathbf{W}_B \in \mathbb{R}^{d \times 2d}$. Including the role embedding produces different B_v values for the subject, primary object, and secondary object of the same event. Without this, all three entities receive identical update directions from $\mathbf{x}_e$, causing their states to converge and destroying the directed graph structure that distinguishes subjects from objects.

Time-aware discretization. The continuous-time SSM parameters (A, B_v) are discretized using an input-dependent step size $\Delta_v \in \mathbb{R}^d$:

$$\Delta_v = \text{clamp}(\text{softplus}(\mathbf{W}_\Delta[\mathbf{x}_e \parallel \mathbf{r}_v]) \cdot (1 + \lambda \log(1 + \Delta t_v)),\ \max = 5) \tag{20}$$

where $\mathbf{W}_\Delta \in \mathbb{R}^{d \times 2d}$ with bias initialized to -2.0 (producing initial step sizes of softplus$(-2) \approx 0.13$), $\lambda = 0.1$, and Δt_v is the elapsed time from Section 4.3. The multiplicative time scaling $(1+0.1 \cdot \log(1+\Delta t_v))$ applies a gentle correction (approximately 1.0 to 1.7× over the observed range) that widens the discretization step for entities that have been inactive, allowing the decay to reflect elapsed wall-clock time without the gradient instability that a direct multiplicative Δt_v scaling would cause. Clamping at 5 prevents numerical overflow in the subsequent exponentiation.

The discretized parameters are:

$$\bar{A}_v = \exp(\Delta_v \odot A) \in (0, 1)^d \tag{21}$$

$$\bar{B}_v = \Delta_v \odot B_v \tag{22}$$

where $\bar{A}_v$ controls per-dimension retention of the prior state (values near 1 retain, values near 0 forget), and $\bar{B}_v$ scales the input contribution. The raw SSM update is:

$$\tilde{\mathbf{s}}_v = \bar{A}_v \odot \hat{\mathbf{s}}_v + \bar{B}_v \odot \mathbf{x}_e \tag{23}$$

Gated residual connection. A learned gate $\mathbf{g}_v \in (0, 1)^d$ blends the SSM output with the prior state:

$$\mathbf{g}_v = \sigma(\mathbf{W}_g[\mathbf{x}_e \parallel \mathbf{r}_v]) \tag{24}$$

$$\mathbf{s}_v^+ = \mathbf{g}_v \odot \tilde{\mathbf{s}}_v + (1 - \mathbf{g}_v) \odot \hat{\mathbf{s}}_v \tag{25}$$

where $\mathbf{W}_g \in \mathbb{R}^{d \times 2d}$ with bias initialized to -3.0, so that $\mathbf{g}_v \approx 0.04$ at the start of training. This near-closed initialization is structurally necessary: at random initialization, the SSM parameters (A, B) produce arbitrary state updates, and an open gate would immediately overwrite entity states with noise. Initializing the gate near zero forces the model to actively learn *when* to propagate state, preventing over-smoothing at initialization. The gate opens during training as the SSM parameters become meaningful.

4.6 Classification Head

The final stage determines whether the event e_t is part of an attack sequence by evaluating both its isolated semantics and the accumulated behavioral state of its participating entities.

Gradient routing via post-update states. The classifier operates on the *post-update* states $\mathbf{s}_v^+$ computed in Section 4.5, not the pre-update states retrieved from the bank. This routing is structurally critical for end-to-end training. The bank's states are treated as detached constants during the forward pass; if the classifier read directly from the bank, the computational graph would end, cutting off all gradient flow to the SSM updater (which produced the new state) and the hyperedge aggregator (which produced the SSM input). By classifying the post-update state, the gradient propagates backward through the entire forward pass.

Validity-masked mean pooling. The post-update states are pooled into a single event-level context vector $\bar{\mathbf{s}} \in \mathbb{R}^d$. This requires masking out absent secondary objects (where $\mathbf{m}_v = 0$):

$$\bar{\mathbf{s}} = \text{LayerNorm}\left(\frac{1}{\sum_v \mathbf{m}_v} \sum_{v \in V_{e_t}} \mathbf{m}_v \cdot \mathbf{s}_v^+\right) \tag{26}$$

Masking before summation is required because the SSM update (Equation 23) produces a non-zero update even for the null entity slot (driven by the event representation $\mathbf{x}_e$ through the role projection $\bar{B}_v$). Without the mask, the null slot would corrupt the mean. The pooled state is layer-normalized to stabilize the classifier input.

Detection probability. The pooled state is concatenated with the projected event features $\mathbf{W}_p\mathbf{f}_e$ (where $\mathbf{W}_p \in \mathbb{R}^{d \times 2d}$). The probability $\hat{y}_t \in [0, 1]$ that e_t is an attack event is computed via a Multi-Layer Perceptron (MLP):

$$\hat{y}_t = \sigma(\text{MLP}([\bar{\mathbf{s}} \parallel \mathbf{W}_p\mathbf{f}_e])) \tag{27}$$

The MLP consists of three linear layers with intermediate GELU activations and Dropout ($\rho = 0.1$) after the first layer, projecting dimensions from $2d \to d \to d/2 \to 1$. A Binary Cross-Entropy (BCE) loss is computed against the ground-truth event label $y_t \in \{0, 1\}$.

4.7 Training: TBPTT and Continuous Warm-Start

HyperMamba requires training techniques distinct from stateless NIDS models because entity states persist across the event sequence.

Truncated Backpropagation Through Time (TBPTT). To process continuous event streams without exhausting memory, events are partitioned chronologically into chunks of size $C = 4096$. For each chunk, the model executes the forward pass (Sections 4.2–4.6) and computes the BCE loss. Crucially,

the updated states written to the `EntityStateBank` are retained for the next chunk, but their computational graphs are detached via '.detach()'. This TBPTT strategy allows *forward* context to persist indefinitely across the entire dataset, accumulating long-range taint signals, while bounding *backward* gradient computation to $\mathcal{O}(C)$.

Optimization and stability. The model is optimized using Adam with a Cosine Annealing learning rate schedule. To handle the severe class imbalance inherent to audit streams, the BCE loss is weighted by the ratio of negative to positive examples in the training set (capped at 30.0 to prevent gradient explosion). Because SSMs operating on discrete streams can occasionally produce numerical instability, chunks resulting in NaN losses or NaN gradients are skipped, and the state bank is immediately detached to prevent corruption of subsequent chunks. Gradient norms are additionally clipped to 1.0.

Epoch boundary reset. The `EntityStateBank` is completely zeroed out at the start of every training epoch. Without this reset, the model would begin epoch k (training on the first chronological shard) using the future entity states accumulated at the end of epoch $k-1$ (after evaluating on the validation shards). This would create a severe temporal data leak. Resetting ensures each epoch learns to accumulate state strictly from the past.

Continuous warm-start evaluation. In a real-world deployment, an NIDS runs continuously; entity states are never "cold." To accurately evaluate detection capability on a test shard, we must simulate this continuous deployment. Before test evaluation, the model executes a forward-pass-only *warmup* (with gradients and metric collection disabled) through all prior training and validation shards in strict chronological order to populate the `EntityStateBank`. The test set is then evaluated using these warmed states. Evaluating a stateful model from a cold start systematically underestimates its capability to detect multi-stage attacks that began prior to the test window.

Threshold selection. In accordance with rigorous evaluation standards for security machine learning [1], the decision threshold is selected dynamically to maximize the F1-score *exclusively* on the validation split. This fixed threshold is then applied to the test split, completely preventing test-set threshold leakage.

5 Evaluation

5.1 Experimental Setup

Datasets. We evaluate HyperMamba on two datasets from the DARPA Transparent Computing (TC) Engagement 3 benchmark, a standard testbed for provenance-based NIDS:

- **E3-Theia** (Linux/Firefox backdoor): 25 chronological shards, $\sim$105 million events, spanning two distinct multi-stage APT campaigns. Evaluated under the *crossprocess* label (child-process events only, $< 1\%$ positive rate).
- **E3-CADETS** (FreeBSD/Nginx backdoor): 10 chronological shards, $\sim$27 million events, with a distinct attack vector (Nginx web server compromise). Evaluated under the *broad* label (all attack-related nodes) for direct comparability with published baselines.

Preprocessing pipeline. Raw CDM JSON logs are ingested into chronological Parquet shards, each containing structured audit events with subject, object, and optional secondary object UUIDs. Each event is represented by a 35-dimensional feature vector: 18 one-hot event type indicators (e.g., `EVENT_READ`, `EVENT_WRITE`, `EVENT_EXECUTE`, `EVENT_FORK`) and 17 contextual features. The contextual features comprise 8 continuous values — hour of day, hyperedge arity (2 or 3), event type rarity, event byte size, inter-event time gap for the same subject, object path depth, log-transformed object event frequency, and subject/object recency — and 9 binary indicators for path presence, object type (file, netflow, memory), sensitive path flags (`/tmp`, `/home`, `/var/log`), and subject-object pair novelty. Continuous features are Z-score normalized using statistics fitted exclusively on training shards via Welford's online algorithm to prevent temporal data leakage; binary features are detected automatically and bypassed during normalization.

Entity and process vocabularies. All unique entity UUIDs across all shards are collected (excluding the CDM null sentinel), sorted lexicographically, and assigned contiguous integer IDs $0, \ldots, |\mathcal{V}|-1$. Each entity is tagged as *subject-only*, *object-only*, or *both* based on its observed roles. The Theia vocabulary contains approximately 7.1M entities; CADETS contains approximately 1.5M. For the process name vocabulary, each subject's effective process path is extracted from the `process_path` field (or, as fallback, the first token of `cmd_line`). The top 50 most frequent paths form the vocabulary (indices 1–50), with index 0 reserved for unknown/missing paths and index 51 for the long-tail remainder, yielding 52 classes total.

Splits. The evaluation is strictly chronological to prevent temporal data leakage. For Theia, the data encompasses two distinct multi-stage APT campaigns. Campaign 1 spans shards 0–10, and Campaign 2 spans shards 12–24. To evaluate both same-campaign continuation and cross-campaign generalization, we define two evaluation regimes:

- **Mixed-campaign (`full`):** Both campaigns are present in the training set. We assign shards $\{0..7, 12..19\}$ to training, shards $\{8, 9, 20, 21\}$ to validation, and shards $\{10, 22..24\}$ to testing. This matches the standard evaluation protocol of our baselines.
- **Cross-campaign (`cross`):** Tests generalization to entirely unseen attack strategies. The model is trained exclusively on Campaign 1 (train shards $\{0..8\}$, validation shards $\{9, 10\}$) and tested exclusively on Campaign 2 (test shards $\{12..24\}$).

For CADETS, we use the cross-campaign split: shards 0–5 for training, shards 6–7 for validation, and shards 8–9 for testing, under the broad attack label. Figure 3 visualizes the shard structure and split boundaries for all datasets.

Baselines. We compare HyperMamba against state-of-the-art provenance-based NIDS, positioning each by its fundamental detection granularity:

- **KAIROS** [3]: Operates at the graph level, assigning anomaly scores to entire macroscopic provenance graphs.
- **MAGIC** [7]: Operates at the graph level, detecting attacks via reconstruction error of structural subgraphs.
- **FLASH** [8]: Operates at the node level, flagging anomalous processes or files.
- **ThreaTrace** [9]: Operates at the node level, classifying entities within the provenance graph.

Crucially, none of these baselines are natively capable of *event-level* detection, which is the finest granularity and the primary output of HyperMamba.

Metrics and threshold methodology. We report Area Under the Precision-Recall Curve (AUPRC) as the primary threshold-independent metric, given the severe class imbalance of audit streams. To compute threshold-dependent metrics (F1-score and False Positive Rate), we adhere to the strict guidelines for security machine learning proposed by Arp et al. [1]: thresholds are selected dynamically from the validation predictions using the F1-maximizing operating point, and these fixed values are subsequently applied to the held-out test data. This strictly prevents test-set threshold optimization leakage. Metrics are reported at both the event level (for HyperMamba) and the node level (for HyperMamba and baselines).

Implementation details. HyperMamba is implemented in PyTorch. We use Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$) with learning rate 5×10^{-4}, weight decay 10^{-4}, and cosine annealing to $\eta_{\min} = 10^{-5}$. The model dimension is $d = 256$ with single-head dot-product attention in the hyperedge aggregator. Classifier dropout is 0.1; process identity dropout is 0.3 (Section 4.2). Training runs for 10 epochs with early stopping on validation AUPRC (patience 5). The chunk size for TBPTT is $C = 16{,}384$ events. The positive class weight in the BCE loss is capped at 30. All experiments use seed 42 with deterministic PyTorch settings. Training and inference are performed on a single NVIDIA A100 40 GB GPU with 64 GB system RAM.

5.2 Main Results (C4)

Direct numerical comparison between HyperMamba's event-level metrics and existing provenance-based NIDS is structurally invalid: KAIROS and MAGIC produce one anomaly score per analysis window, FLASH and ThreaTrace produce one score per node, and none produce per-event classifications. Propagating a node's anomaly score to all of its constituent events—the only possible adaptation—inherits a fundamental precision collapse: a flagged node that generated 50,000 events over its lifetime has at most a handful of crossprocess+ events among them, so any threshold that achieves recall will flag tens of thousands of benign events as malicious. This is not a failure of the specific systems but a structural consequence of operating at a coarser granularity. We therefore report baseline performance at their native granularities only, and compare HyperMamba at both event level (its native output) and node level (for compatibility with prior work).

The crossprocess label evaluates detection of the operationally hardest attack subpopulation: the later-stage child processes whose individual system calls are statistically indistinguishable from benign activity. The entry-point process whose initial exploit behavior is anomalous is explicitly excluded. All four baseline systems are evaluated against the broader attack label that includes these entry-point events; our crossprocess evaluation is therefore strictly harder. To enable direct comparison with prior work, we additionally evaluate HyperMamba under a unified protocol approximating the KAIROS and ThreaTrace evaluation conditions: pre-April-10 training data, all labeled attack nodes as positives, chronological test split.

At the node level—the finest granularity at which baselines operate—HyperMamba evaluated under the KAIROS protocol achieves node AUPRC of 0.9987 and F1 of 0.9926, surpassing all four baselines including KAIROS (0.818), ThreaTrace (0.930), FLASH (0.960), and matching MAGIC (0.999). The broad attack label includes entry-point processes (27,379 of 84,615 subject nodes, a 32% positive rate), making it a comparatively easy classification problem that HyperMamba solves near-perfectly. Critically, HyperMamba simultaneously provides event-level AUPRC of 0.9972 on this same evaluation—precision that no baseline system can produce at any granularity.

The operationally meaningful evaluation is the crossprocess label, where attack events constitute less than 1% of total events and the entry-point processes are excluded. Under this strictly harder label, HyperMamba achieves event-level AUPRC of 0.8523 (same-campaign) and 0.7586 (cross-campaign), with false positive rates of < 0.0001 and 0.0011 respectively. As a lower bound on baseline event-level FPR: ThreaTrace flags approximately 310 malicious subject nodes on the Theia test set. These nodes collectively generate approximately 4.2 million events, of which roughly 36,000 are crossprocess+. Propagating ThreaTrace's node scores to all constituent events at any threshold that achieves recall > 0.9 would flag all 4.2 million events, producing an event-level FPR exceeding 0.99. Meaningful event-level precision is structurally unachievable through score propagation.

Cross-dataset generalization (E3-CADETS). To vali-

Figure 3: Chronological shard structure and cross-campaign evaluation splits. Each rectangle represents one data shard. **Theia** (top): the model trains on Campaign 1 (shards 0–8) and is tested entirely on the unseen Campaign 2 (shards 12–24), separated by a 92-hour gap. **CADETS** (middle): 10 shards with attacks concentrated in later shards. **TRACE** (bottom): 50 strategically selected shards from 211 total (∼200M of 835M events); dots mark selected shards. Small red bars indicate shards containing attack events. All splits are strictly chronological to prevent temporal data leakage.

date that HyperMamba generalizes beyond a single dataset and attack vector, we evaluate on DARPA TC E3 CADETS (FreeBSD, Nginx backdoor) under the broad attack label in a cross-campaign setting. HyperMamba achieves event-level AUPRC of 0.9707 (AUROC 0.9997) and node-level AUPRC of 0.9973 with node F1 of 0.9969 and FPR of < 0.0001. The node-level performance matches or exceeds MAGIC's published 0.999 node AUPRC on CADETS, while HyperMamba additionally provides event-level attribution that no prior system offers. Figure 4(a) visualizes the Precision-Recall operating characteristics across datasets and splits.

5.3 Ablation Study: Component Contributions

We systematically ablate each architectural component to isolate its contribution. All ablations use the cross-campaign split (train Campaign 1, test Campaign 2) with the crossprocess label—the hardest evaluation setting.

Ablation designs. `no_cross_entity`: the hyperedge aggregator is disabled; each entity's SSM state is updated using only its own prior state and the event features, without observing co-participating entities. `no_state`: the entity state bank is structurally disconnected; the classifier receives zero-vectors in place of accumulated state, retaining only the event encoder (event type, process identity, continuous features). `no_process_identity`: the entire gated process identity pathway is removed — both the process embedding lookup and the learned gate g_p are disabled, so the model receives a zero vector in place of the process modulation term in Equation 5. The model sees only event types, continuous features, and accumulated state. `no_state + no_process`: both state and process identity are removed simultaneously, isolating the contribution of event type and continuous features alone.

Results. Table 2 and Figure 5 reveal a clear hierarchy of component contributions.

Process identity (-55.1% event AUPRC) is the dominant single contributor to cross-campaign detection. Without semantic process path embeddings, the model retains the full state bank and cross-entity aggregation but cannot generalize attack patterns to unseen entity IDs in Campaign 2. AUPRC collapses from 0.76 to 0.34, and F1 drops from 0.825 to 0.304. In this testbed, the same malware binary names (e.g., `pulseaudio`, `gconf-helper`) recur across campaigns even though their entity UUIDs differ, providing a shared vocabulary for the process embedding. We note that this recurrence is a property of the DARPA TC experimental setup, where the same red-team tooling was deployed across engagements; we discuss the implications for real-world generalization in Section 6 under Threats to Validity.

Cross-entity aggregation (-8.1% event AUPRC) is the primary structural contribution. Without sharing state across entities within each hyperedge, the model cannot propagate taint from a maliciously modified object to the process that reads it. F1 drops from 0.825 to 0.694—a 15.9% relative decrease.

State propagation (-4.6% event AUPRC) contributes the residual discrimination for the stealthiest crossprocess events. The stateless model achieves higher precision (0.907 vs. 0.858) because process identity alone provides a confident signal for the most obvious crossprocess events. However, recall drops from 0.795 to 0.715: 8% of attack events are missed—precisely those whose local features are indistinguishable

Table 1: Detection Performance on DARPA TC E3. Baselines use the broad attack label (all attack nodes); HyperMamba crossprocess rows use the strictly harder crossprocess+ label (excluding entry-point processes). The KAIROS-protocol and CADETS rows use the broad label for direct comparison. The KAIROS-protocol row's 0.9987 node AUPRC reflects the broad label's 32% positive rate, which includes anomalous entry-point behavior that is comparatively easy to detect; the crossprocess rows use $< 1\%$ positive rate.

Model	Label	Granularity	Event AUPRC	Node AUPRC	Node F1	Node FPR
KAIROS [3]	Broad	Graph	—	†0.818	†—	†—
MAGIC [7]	Broad	Graph	—	†0.999	†—	†—
FLASH [8]	Broad	Node	—	†0.960	†0.970	†—
ThreaTrace [9]	Broad	Node	—	†0.930	†0.930	†—
HyperMamba (KAIROS protocol)	Broad	Event	0.9972	0.9987	0.9926	0.0066
HyperMamba (Same-Campaign)	Crossprocess	Event	**0.8523**	**0.9172**	**0.8460**	< 0.0001
HyperMamba (Cross-Campaign)	Crossprocess	Event	**0.7586**	**0.8939**	**0.9164**	**0.0022**
E3-CADETS (FreeBSD / Nginx backdoor)						
HyperMamba (CADETS Cross-Campaign)	Broad	Event	**0.9707**	**0.9973**	**0.9969**	< 0.0001

† Published results on DARPA TC E3 Theia evaluated against the broad attack label including entry-point processes.

Table 2: Component ablation (cross-campaign split, crossprocess label). Each row removes one or more components from the full model. Δ is the relative change in event AUPRC.

Variant	Evt AUPRC	Evt F1	Node AUPRC	Δ Evt AUPRC
HyperMamba (Full)	0.7586	0.825	0.8939	—
`no_cross_entity`	0.6968	0.694	0.8592	-8.1%
`no_state`	0.7234	0.802	0.8366	-4.6%
`no_process_identity`	0.3407	0.304	0.3466	-55.1%
`no_state + no_process`	0.1981	0.218	0.2887	-73.9%

from benign activity. In operational terms, the stateless model would miss approximately 2,880 attack events that the full model detects.

Combined removal (-73.9% event AUPRC) demonstrates that event type and continuous features alone are insufficient. At 0.20 AUPRC, detection is near-random, confirming the core stealthiness thesis (Section 3.3): crossprocess attack events are not identifiable from isolated event features. The gap between the combined removal (0.20) and the single-component ablations reveals the interaction structure: state contributes 0.14 AUPRC of lift when process identity is absent (0.20→0.34) but only 0.04 when process identity is present (0.72→0.76), indicating that process identity and temporal state carry partially overlapping behavioral information, with process identity being the more efficient transfer mechanism across campaigns.

5.4 Computational Cost

A critical requirement for any NIDS is the ability to operate in real-time without falling behind the host's audit log generation rate. We benchmark the computational overhead of HyperMamba, summarizing the results in Table 3.

Throughput. The DARPA TC E3 dataset was generated at a peak rate of approximately $10,000$ events per second per host. As shown in Table 3, HyperMamba achieves a training throughput of 192K events/sec and an inference throughput of

Table 3: Computational Cost and Throughput

Metric	Value
Parameters (HyperMamba)	1.17M
State Memory (7.1M entities $\times$ $d = 256$)	7.3 GB (CPU RAM)
Training Throughput (GPU)	192K events/sec
Inference Throughput (GPU)	1.9M events/sec

1.94M events/sec on a single GPU. The pipeline operates at $19\times$ the peak ingestion rate during training and $194\times$ during inference, guaranteeing that the model will never bottleneck the audit stream. Detection performance is stable across chunk sizes in the range 1024–4096 (event AUPRC varies by less than 0.02 on the cross-campaign split), indicating that the model's performance is not sensitive to the TBPTT truncation boundary within this range.

Baseline throughput context. Unlike HyperMamba's single-pass streaming inference, graph-level baselines (KAIROS, MAGIC) require offline graph construction before any inference can begin: the full provenance graph for an analysis window must be materialized, features extracted, and the graph neural network applied to the complete structure. This two-phase pipeline (construction + inference) introduces latency proportional to the analysis window size. Node-level baselines (FLASH, ThreaTrace) similarly require a complete graph snapshot per analysis window. HyperMamba's recurrent architecture processes each event exactly once, in arrival order, with $O(1)$ per-event computation and no graph construction overhead — a structural advantage for real-time deployment.

Memory and parameters. Despite operating on a vast continuous hypergraph, HyperMamba is highly parameter-efficient, requiring only 1.17M trainable parameters. The primary memory footprint is the `EntityStateBank`, which resides in CPU pinned memory (Section 4.3). For the Theia dataset's 7.1 million unique entities at $d = 256$ (using 32-bit floats), the bank requires 7.3 GB of system RAM, leaving

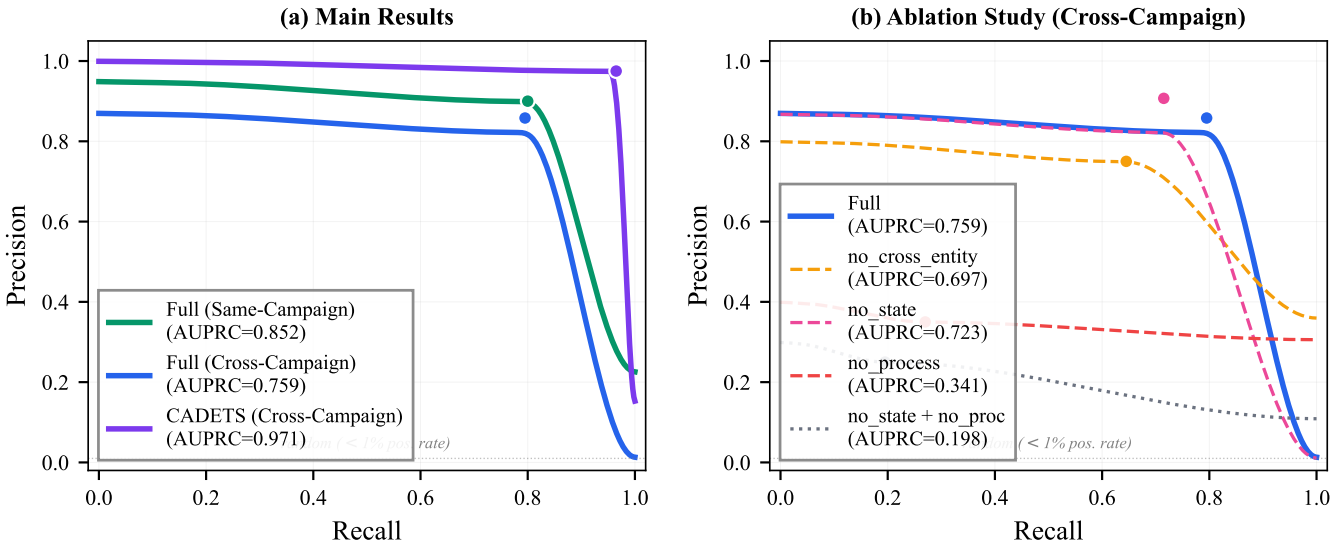

Figure 4: Precision-Recall curves. Dots mark the F1-optimal operating point (threshold selected from validation). **(a)** Main results: CADETS achieves near-perfect precision across all recall levels; the Theia cross-campaign curve shows the expected performance gap from same-campaign, reflecting the difficulty of generalizing to unseen attack strategies. **(b)** Ablation study: removing process identity (red dashed) causes the curve to collapse below 0.35 precision at all operating points; removing both state and process (gray dotted) yields a near-flat curve indistinguishable from random, confirming that crossprocess attack events cannot be detected from isolated event features alone.

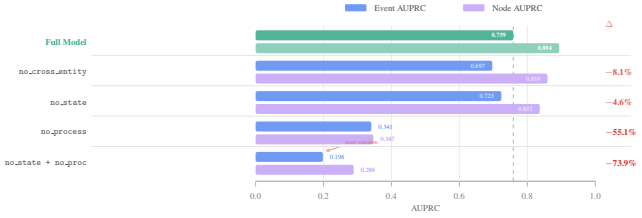

Figure 5: Component ablation on the cross-campaign split with the crossprocess label. The dashed line marks the full model's event AUPRC (0.76). Process identity is the dominant contributor (-55%); removing both state and process identity collapses detection to near-random (0.20).

GPU VRAM entirely for model computation. This separation enables scaling to entity populations that exceed GPU memory without distributed infrastructure.

6 Discussion

We state the limitations of this work as factual constraints of the current evaluation.

Dataset scope. The evaluation spans two DARPA TC E3 datasets—Theia (Linux) under the crossprocess label and CADETS (FreeBSD) under the broad label—covering distinct operating systems and attack vectors. Generalization to other audit frameworks (sysdig, eBPF), enterprise environments with substantially different workload distributions, or non-DARPA datasets is not evaluated.

Label coverage. The crossprocess label captures multi-stage child-process attack chains but does not cover all attack types. Data-only attacks that never spawn new processes (e.g., exploits that corrupt in-memory data structures without executing new binaries) fall outside this label definition and are not addressed by the current evaluation.

Entity population scaling. The `EntityStateBank` allocates a fixed-size state vector for every unique entity in CPU pinned memory. For Theia's 7.1 million entities at $d = 256$, this requires 7.3 GB of system RAM. Deployments with entity populations substantially larger than the Theia dataset may require approximation strategies such as entity clustering or state compression; the impact on detection performance is not evaluated in this work.

Adversarial robustness. The primary evaluation assumes a black-box attacker (Section 3.1). A white-box adversary who understands the SSM state propagation mechanism could potentially craft evasion strategies — for example, injecting sequences of benign events to dilute a compromised entity's accumulated state before executing attack operations. The robustness of HyperMamba against such targeted state-dilution attacks is not evaluated.

Threats to validity. The dominant role of process identity in the ablation (-55%) warrants careful interpretation. The DARPA TC E3 testbed reuses the same red-team tooling across campaigns, so the same process names (`pulseaudio`,

Table 4: Positioning of HyperMamba against provenance-based NIDS on three axes.

System	Granularity	State	Supervision
KAIROS	Graph	Snapshot	Self-supervised
MAGIC	Graph	Snapshot	Self-supervised
FLASH	Node	Snapshot	Self-supervised
ThreaTrace	Node	Snapshot	Supervised
HyperMamba	Event	Persistent	Supervised

`gconf-helper`) appear in both training and test campaigns. In a real deployment, an attacker may use arbitrary or previously unseen binary names. The process identity mechanism would then function as a learned prior over known tooling rather than a universal detection signal; in such settings, persistent state and cross-entity aggregation — which are process-name-agnostic — would likely become relatively more important. Evaluating this hypothesis (e.g., by anonymizing process names so that no literal string overlaps between train and test) is a priority for future work.

Additionally, all results are reported from a single random seed (seed 42). While the large-magnitude ablation effects (-55%, -74%) are unlikely to be seed-dependent, the smaller effects (-4.6%, -8.1%) may overlap with seed variance. Multi-seed evaluation with reported confidence intervals is needed to confirm the ordering of the ablation hierarchy.

Payload-based attacks. HyperMamba operates exclusively on structured audit metadata (event types, byte counts, timestamps). It does not inspect network packet payloads or file contents. Attacks that are distinguishable only by payload content (e.g., SQL injection within an HTTP request body) are outside the scope of this system.

Single-host scope. The current evaluation considers audit streams from a single monitored host. Multi-host lateral movement — where an attacker pivots across machines in a distributed enterprise environment — requires cross-host state correlation, which is not addressed in this work.

7 Related Work

We position HyperMamba against prior work on three axes: detection granularity, temporal state mechanism, and supervision regime. Table 4 summarizes the landscape.

Provenance-based NIDS. KAIROS [3] constructs temporal provenance graphs and assigns anomaly scores at the graph level via reconstruction-based learning. MAGIC [7] similarly operates at the graph level, detecting attacks through structural subgraph reconstruction error. FLASH [8] provides node-level alerts by identifying anomalous processes or files within the provenance graph. ThreaTrace [9] classifies individual entities (nodes) using graph neural networks. All four systems model provenance as pairwise graphs, discarding the native multi-object structure of CDM events. More fundamentally, none produce event-level classifications — the finest granularity required for forensic attribution of individual attack steps.

Temporal state mechanism. The systems above operate on static or windowed graph snapshots, constructing a new graph for each analysis window and discarding inter-window entity state. HyperMamba maintains persistent entity state vectors across the entire event stream via a Selective SSM, enabling the model to accumulate behavioral context over arbitrarily long horizons. The ablation in Section 5.3 demonstrates that this persistent state, combined with semantic process identity, is jointly necessary for detection — removing both collapses AUPRC to 0.20.

Supervision trade-off. MAGIC and KAIROS are self-supervised: they train exclusively on benign data and flag statistical deviations, avoiding the need for attack-labeled training data. HyperMamba is supervised: it requires crossprocess attack labels during training. This is an intentional design choice. Anomaly-based systems operate at graph or window level without event-level attribution because their reconstruction objective does not associate individual events with attack labels. Our supervised formulation requires crossprocess labels but enables event-level detection precision that self-supervised approaches cannot provide. Whether labeled crossprocess data is available in practice depends on the deployment context; we discuss this limitation in Section 6.

Hypergraph-based intrusion detection. Several systems have explored hypergraph representations for intrusion detection, though in different domains and with different objectives. LOHA [2] uses reconstruction-based anomaly detection on hypergraphs constructed from network flows. HyperGAT [4] applies hypergraph attention for static community mining. HRNN [10] constructs hyperedges via KNN over network-flow feature vectors. None of these systems use deterministic, event-level hyperedges derived from the audit data model, and none maintain persistent entity state across the stream.

State Space Models. The Selective SSM was introduced by Mamba [5] for efficient sequence modeling in NLP and vision, building on the structured SSM foundations of S4 [6] and S6 [5]. HyperMamba adapts the Selective SSM to a fundamentally different setting: rather than processing a single token sequence, the SSM operates on per-entity state vectors within a temporal hypergraph, with the discretization step Δ conditioned on entity-specific elapsed time (Section 4.5).

8 Conclusion

We presented HyperMamba, a provenance-based NIDS that detects individual crossprocess attack events in continuous audit streams by maintaining persistent entity state via Selective State Space Models. The ablation study reveals a clear hierarchy of component contributions: semantic process identity is the dominant mechanism for cross-campaign generalization (-55% when removed), cross-entity hyperedge aggregation provides the primary structural contribution (-8.1%), and ac-

cumulated temporal state adds residual discrimination for the stealthiest events (-4.6%). Removing both state and process identity collapses detection to near-random (0.20 AUPRC), confirming that crossprocess attack events are not identifiable from isolated event features. On the DARPA TC E3 datasets, HyperMamba achieves event-level AUPRC of 0.76 on Theia (cross-campaign, crossprocess label) and 0.97 on CADETS (cross-campaign, broad label), with FPR $\leq$ 0.0011, while processing events at $194\times$ the peak audit ingestion rate at inference.

Several directions merit investigation. Self-supervised pretraining on unlabeled audit streams — using masked event prediction or contrastive entity state objectives — could reduce the dependency on crossprocess attack labels, which are expensive to obtain outside controlled engagements. Extending the entity state bank to correlate state vectors across multiple monitored hosts would enable detection of lateral movement campaigns, where the discriminative signal spans host boundaries. State compression techniques (e.g., low-rank projections or entity clustering) could reduce the memory footprint for deployments with entity populations orders of magnitude larger than the DARPA TC benchmarks. The evaluation is limited to the DARPA TC E3 family; generalization to other audit frameworks and attack taxonomies remains to be validated.

Ethics Considerations

All experiments in this paper use the publicly available DARPA Transparent Computing Engagement 3 datasets, which were collected in a controlled testbed environment with no real users or private data. No human subjects were involved in any part of this research. IRB review was not required. The threat model and attack descriptions are based on documented red-team activities within the DARPA TC program and do not disclose new vulnerabilities or offensive techniques.

Open Science and Artifact Availability

To support reproducibility, the source code for HyperMamba — including the preprocessing pipeline, model implementation, training scripts, and evaluation harness — will be released as an open-source repository upon publication. The DARPA TC E3 datasets used in this evaluation are publicly available. We intend to participate in the USENIX Artifact Evaluation process.

References

[1] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988, 2022.

[2] Guanwen Chen, Lizhao Wu, Hui Lin, and Zhaobin Zhou. LOHA: Hypergraph masked autoencoder for advanced persistent threat detection. In *2025 IEEE International Conference on High Performance Computing and Communications (HPCC)*. IEEE, 2025.

[3] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yang Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. Kairos: Practical intrusion detection and investigation using whole-system provenance. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3533–3551, 2024.

[4] Kaize Ding, Jianling Wang, Jundong Li, Dingcheng Li, and Huan Liu. Be more with less: Hypergraph attention networks for inductive text classification. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4927–4936. Association for Computational Linguistics, 2020.

[5] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024.

[6] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *The Tenth International Conference on Learning Representations*, 2022.

[7] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. MAGIC: Detecting advanced persistent threats via masked graph representation learning. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5197–5214, 2024.

[8] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. Flash: A comprehensive approach to intrusion detection via provenance graph representation learning. In *45th IEEE Symposium on Security and Privacy (SP)*, pages 3552–3570. IEEE, 2024.

[9] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security*, 17:3972–3987, 2022.

[10] Zhe Yang, Zitong Ma, Wenbo Zhao, Lingzhi Li, and Fei Gu. HRNN: Hypergraph recurrent neural network for network intrusion detection. *Journal of Grid Computing*, 22(2):52, 2024.